Pipeline de Dados com IoT e Docker

Disruptive Architectures: IoT, Big Data e IA

Aluno: Lucas Araujo Saro
RA: 127714
Curso: Análise e Desenvolvimento de Sistemas - UniFECAF

Repositório: **https://github.com/Luc14saro/Lucas-Araujo-Saro-bigdatafecaf**

1. Contextualização do projeto

Dispositivos IoT geram continuamente grandes volumes de leituras, como a temperatura de ambientes. Esses dados chegam brutos, em arquivos ou fluxos, e sozinhos dizem pouco: é preciso armazená-los de forma organizada, consultá-los com eficiência e apresentá-los de modo que uma pessoa consiga interpretar e agir.

O problema abordado neste trabalho é justamente esse: como transformar leituras de temperatura de sensores em informação útil. A proposta é um **pipeline de dados** completo, com as etapas de ingestão, armazenamento, consulta e visualização.

- **Ingestão:** script Python lê o CSV do dataset *Temperature Readings: IoT Devices* (Kaggle), limpa e converte os tipos.
- **Armazenamento:** os dados são gravados em um banco PostgreSQL executado em um contêiner Docker, o que torna o ambiente reproduzível.
- **Consulta:** três views SQL resumem os dados de forma reutilizável.
- **Visualização:** um dashboard Streamlit com gráficos interativos Plotly consome as views.

O uso de Docker evita problemas de instalação e de versão do banco, e o uso de views mantém a lógica de agregação dentro do banco, deixando o dashboard simples.

2. Passos realizados

2.1 Configuração do ambiente

Foram instalados Python, Docker e Git. Em seguida foi criado um ambiente virtual e instaladas as dependências:

```
python -m venv .venv
source .venv/bin/activate
pip install pandas psycopg2-binary sqlalchemy streamlit plotly
```

2.2 Criação do contêiner PostgreSQL

O banco foi definido em um arquivo `docker-compose.yml` (imagem `postgres:16`, porta 5432, volume persistente e healthcheck):

```
docker compose up -d
docker compose ps
```

O status **healthy** confirma que o banco está pronto para receber os dados (captura na seção 2.4).

2.3 Download da base de dados

O CSV foi baixado do Kaggle (atulanandjha/temperature-readings-iot-devices) e salvo em `data/IOT-temp.csv`. As colunas originais são `id`, `room_id/id`, `noted_date`, `temp` e `out/in`.

2.4 Inserção no banco

O script `src/ingest.py` usa pandas para ler o CSV, renomeia as colunas, converte a data (formato dia-mês-ano hora:minuto) e a temperatura, remove linhas nulas ou duplicadas e carrega os dados com SQLAlchemy na tabela `temperature_readings`. Ao final, cria as views.

```
python src/ingest.py
```

Resultado: **97.606** linhas lidas do CSV, 1 linha duplicada removida e **97.605** registros carregados no PostgreSQL, com as 3 views criadas.

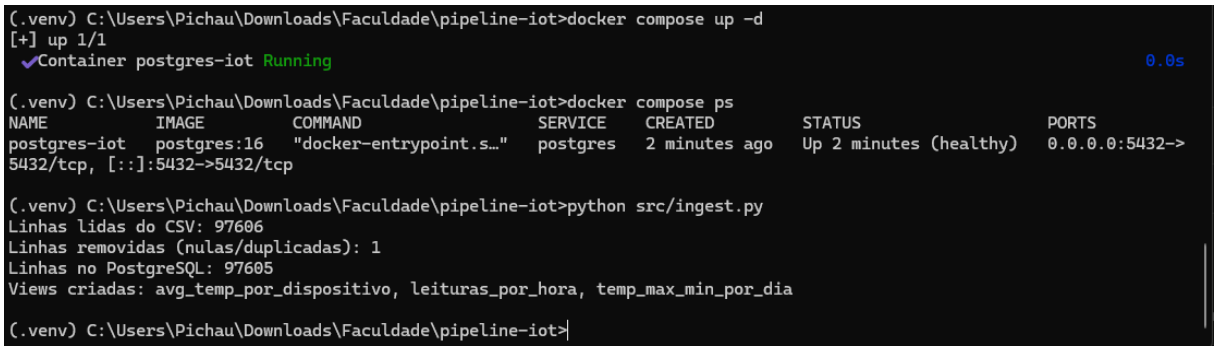


Figura 1 - Container PostgreSQL saudável (docker compose ps) e execução do script de ingestão

A carga foi verificada com uma contagem de registros no PostgreSQL:

```
docker exec -it postgres-iot psql -U iot_user -d iot_db \
  -c "SELECT COUNT(*) FROM temperature_readings;"
```

2.5 Criação das views SQL

As três views estão em `sql/views.sql` e são detalhadas na seção 3.

2.6 Construção do dashboard

O arquivo `src/dashboard.py` usa Streamlit e Plotly para ler cada view com `pandas.read_sql` e exibir três gráficos interativos, além de indicadores gerais (total de leituras, temperatura máxima e mínima).

```
streamlit run src/dashboard.py
```

3. As 3 views SQL

3.1 avg_temp_por_dispositivo

```
CREATE OR REPLACE VIEW avg_temp_por_dispositivo AS
SELECT device_id, location,
       ROUND(AVG(temperature)::numeric, 2) AS avg_temp,
       COUNT(*) AS total_leituras
FROM temperature_readings
GROUP BY device_id, location;
```

Propósito: calcular a temperatura média de cada dispositivo, separada por local da medição (interno ou externo). Permite comparar sensores e medir a diferença térmica entre dentro e fora. Alimenta o gráfico de barras.

3.2 leituras_por_hora

```
CREATE OR REPLACE VIEW leituras_por_hora AS
SELECT EXTRACT(HOUR FROM noted_date)::int AS hora,
       COUNT(*) AS contagem
FROM temperature_readings
GROUP BY EXTRACT(HOUR FROM noted_date)
ORDER BY hora;
```

Propósito: contar quantas leituras chegam em cada hora do dia. Mostra os horários de maior e menor coleta e ajuda a identificar lacunas ou falhas de envio dos dispositivos. Alimenta o gráfico de linha.

3.3 temp_max_min_por_dia

```
CREATE OR REPLACE VIEW temp_max_min_por_dia AS
SELECT noted_date::date AS data,
       MAX(temperature) AS temp_max,
       MIN(temperature) AS temp_min
FROM temperature_readings
GROUP BY noted_date::date
ORDER BY data;
```

Propósito: registrar a temperatura máxima e mínima de cada dia, evidenciando a amplitude térmica diária e destacando dias com picos ou quedas fora do normal. Alimenta o gráfico de linhas com duas séries.

4. Visualizações no Streamlit

Capturas de tela do dashboard com os dados reais, com gráficos interativos Plotly (zoom, tooltip e filtro por série).

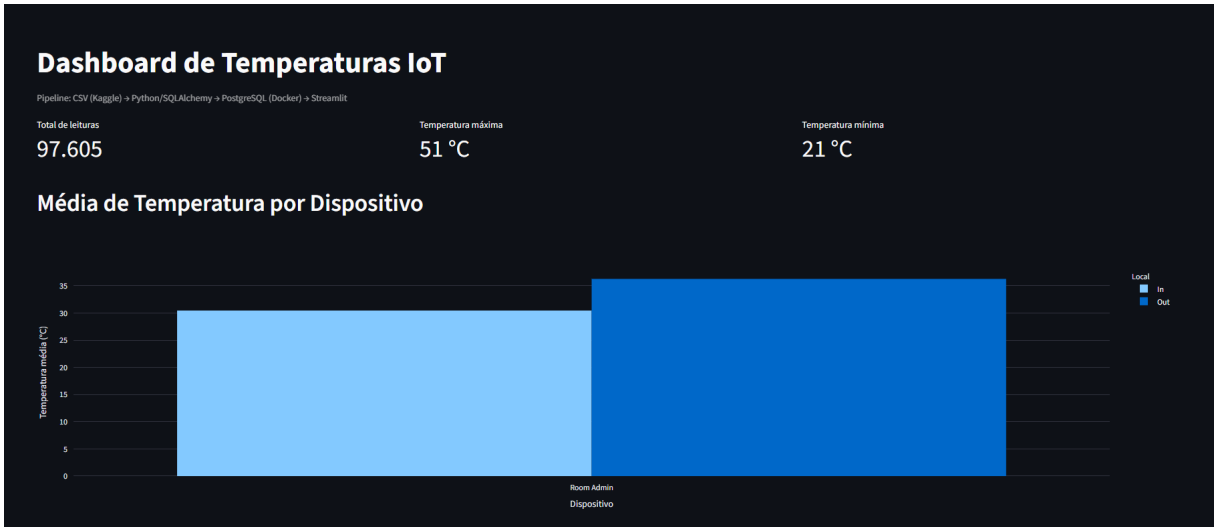


Figura 2 - Média de temperatura por dispositivo (interno x externo)

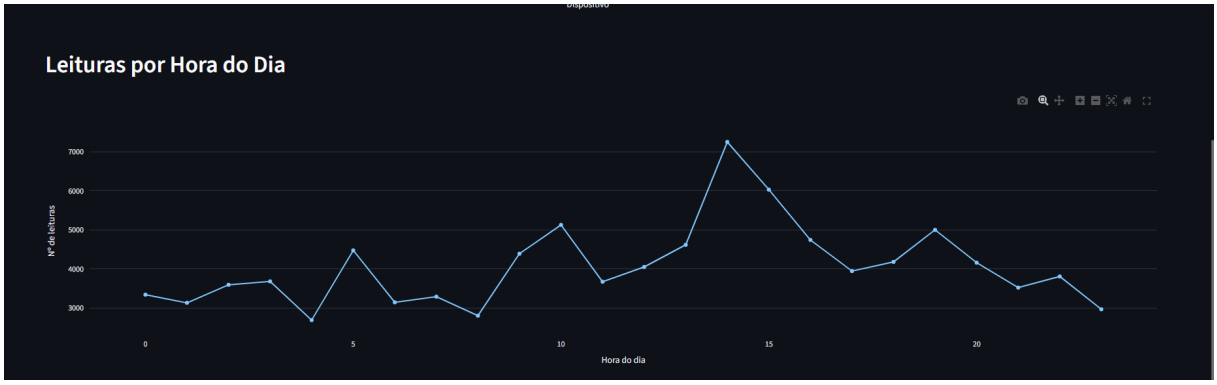


Figura 3 - Quantidade de leituras por hora do dia

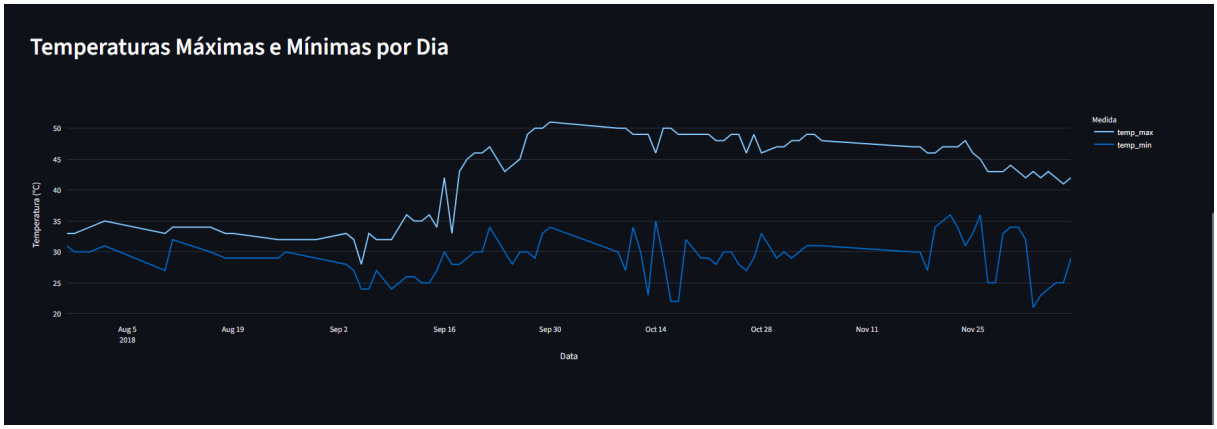


Figura 4 - Temperaturas máximas e mínimas por dia

5. Principais insights e usos práticos

5.1 Insights obtidos

Resultados das consultas de `sql/insights.sql` sobre os 97.605 registros carregados:

- **Visão geral:** leituras entre 28/07/2018 e 08/12/2018, com temperatura média de 35,05 °C, mínima de 21 °C e máxima de 51 °C;

- **Interno x externo:** a média externa (36,27 °C) é 5,82 °C maior que a interna (30,45 °C). Cerca de 79% das leituras (77.260) são externas e 21% (20.345) são internas;
- **Horários de coleta:** o pico de leituras ocorre às 14h (7.248), seguido de 15h (6.029) e 10h (5.127). As menores coletas são às 4h (2.690), 8h (2.803) e 23h (2.967), cerca de 2,7 vezes menos que o pico. Isso mostra um volume de dados bem concentrado no período da tarde;
- **Amplitude diária:** o dia de maior variação foi 16/10/2018, de 22 °C a 50 °C (28 °C de amplitude). Os quatro dias de maior amplitude são todos de outubro (10, 13, 16 e 17);
- **Evolução no período:** as máximas diárias ficavam em torno de 33 a 35 °C em agosto e passam a ficar entre 45 e 51 °C a partir do fim de setembro. Os dados não permitem afirmar a causa (mudança de estação, posição do sensor ou outro fator).

5.2 Sugestões de uso prático em ambiente real

- **Alertas automáticos:** disparar notificação quando a temperatura sair de uma faixa segura (salas de servidores, câmaras frias, estufas, laboratórios);
- **Manutenção preditiva:** detectar aquecimento anormal que antecede falhas de equipamentos;
- **Eficiência energética:** ajustar climatização com base na diferença entre ambiente interno e externo;
- **Monitoramento de qualidade da coleta:** identificar sensores que deixam de enviar dados em certos horários;
- **Escalabilidade:** a mesma arquitetura pode receber dados em tempo real (por exemplo via MQTT) e crescer para milhares de dispositivos.

6. Conclusão

O projeto integrou conceitos de IoT, Big Data e engenharia de dados em um pipeline funcional: ingestão com Python, armazenamento em PostgreSQL via Docker, consultas padronizadas por views SQL e visualização interativa com Streamlit. O código, o Docker e as instruções de execução estão disponíveis no repositório público do GitHub.